ICLR 2024 | A Bigger Graph Network Won't Give You Trustworthy Confidence

University of Michigan and Lawrence Livermore National Laboratory introduce G-ΔUQ, a single-model, scalable way to estimate epistemic uncertainty in graph neural networks

Graph neural networks (GNNs) are increasingly being handed high-stakes jobs: predicting molecular properties, screening materials, flagging anomalies. These settings share an uncomfortable feature: the data a model sees at test time almost never matches what it trained on. Graphs grow larger, their structure drifts, and the relationships between labels and attributes shift. In those moments we care about more than whether the model is right; we care about whether its confidence is right. Downstream safety metrics, from calibration error to out-of-distribution (OOD) rejection to generalization-gap estimation, all sit on top of that confidence indicator, which usually comes from a softmax probability or an entropy score.

The computer vision community has known for years that confidence quality decays under distribution shift. For graph neural networks, how and why confidence degrades has been studied far less. This paper (ICLR 2024) fills that gap systematically, and then offers a direct fix.

A natural first instinct is to reach for a more expressive architecture and assume better calibration comes along for free. Through a controlled experiment, the authors show it does not. Figure 1 makes the counterintuitive result hard to miss.

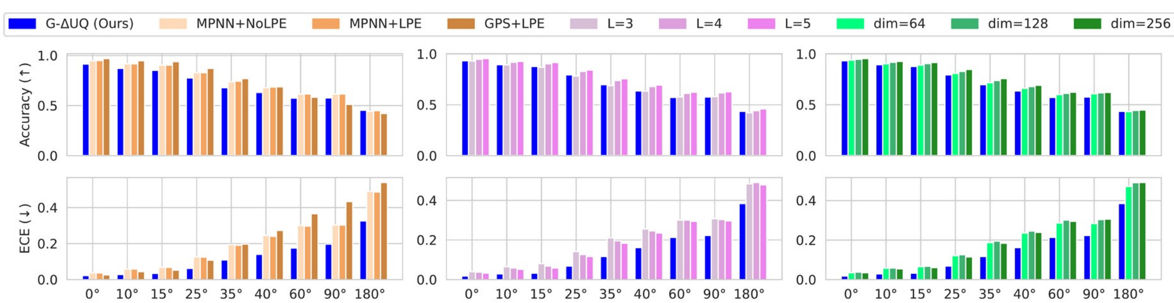

Figure 1. Calibration under structural distortion shift. Models are trained on standard super-pixel MNIST and then evaluated on k-nearest-neighbor graphs built from rotated MNIST images, using the rotation angle to dial the structural distortion from mild to severe. The top row is accuracy (higher is better); the bottom row is expected calibration error, ECE (lower is better). As the shift grows, the accuracy drop is expected, but ECE deteriorates sharply too, and switching to a graph transformer (GPS), adding positional encodings (LPE), or making the network deeper and wider does not rescue it. Only the proposed G-ΔUQ (blue) drives ECE down substantially without materially sacrificing accuracy.

In other words, piling on expressivity is not the cure for miscalibration, and scaling depth or width can even make it worse. What is actually needed is to model epistemic uncertainty explicitly. That is where the paper begins.

Paper and code

Paper: https://arxiv.org/abs/2309.10976

Code: https://github.com/pujacomputes/gduq

Project page: https://pujacomputes.github.io/gduq/

What the paper contributes

The contribution comes in three escalating steps. First, a controlled case study establishing that improving a GNN's expressivity does not fix confidence calibration under distribution shift. Second, G-ΔUQ, a single-model uncertainty method that ports the stochastic centering (anchoring) framework to graph data and, crucially, supports partial stochasticity (making only part of the network stochastic), including a variant that reuses a pretrained backbone as-is. Third, an extensive evaluation across three types of distribution shift and three safety-critical tasks.

The single-model point is worth dwelling on. Unlike Deep Ensembles, which train many models and average them, G-ΔUQ needs just one model to produce both a prediction and an estimate of its uncertainty.

How G-ΔUQ works

Start with the method at a glance. The core idea is anchoring: instead of feeding the model the raw input, you first draw a random anchor c from the training distribution and rewrite the input as a relative representation: subtract the anchor, then concatenate it back, i.e. [X − c ‖ c]. Each iteration uses a different anchor, so the model behaves as if it were sampling from a family of hypotheses. Run the same sample through K different anchors, and the variance of those predictions captures epistemic uncertainty.

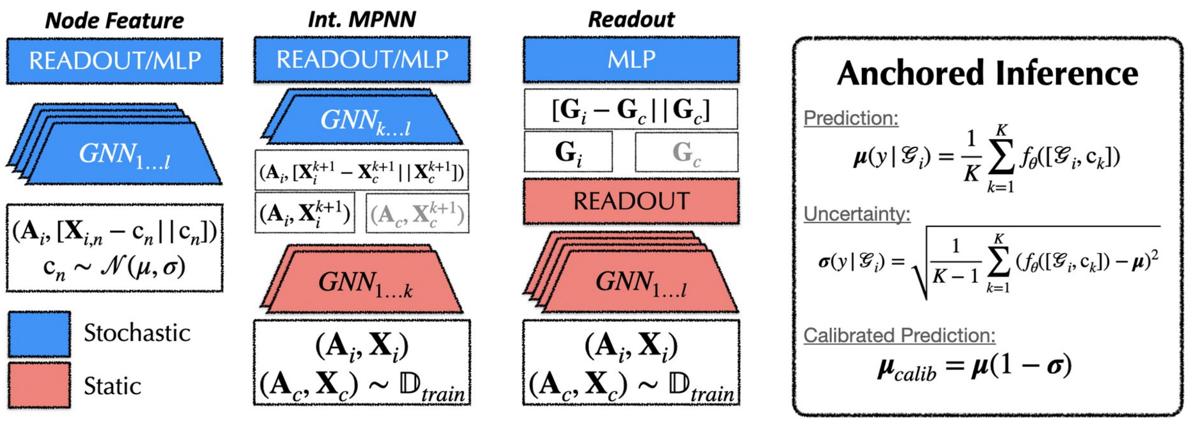

Figure 2. Overview of G-ΔUQ. The authors propose three stochastic centering variants that inject increasing-to-decreasing amounts of stochasticity into the GNN: anchoring on the node features (Node Feature), at an intermediate message-passing layer (Int. MPNN), or after the READOUT (Readout). Blue marks the stochastic part, red the static part. The READOUT variant makes only the post-readout head stochastic, so it can wrap a pretrained backbone directly. On the right is anchored inference: average over K anchors for the prediction μ, use the standard deviation σ across anchors as the uncertainty, and report the calibrated prediction μ_calib = μ(1 − σ).

Why anchor in feature space rather than on the graph structure itself? Subtracting an "anchor graph" from an adjacency matrix would invent edges that do not exist and corrupt the graph's meaning. So the authors anchor in node-feature or hidden-representation space instead. The three variants correspond to injecting the anchor early, in the middle, or late; the later the injection, the smaller and lighter the stochastic part. The variant that freezes a pretrained backbone and trains only an anchored classifier head adds uncertainty to an existing model at almost no cost. That is what partial stochasticity buys you in practice.

Results across three kinds of shift

The evaluation is deliberately broad. Structural distortion uses the rotated super-pixel MNIST benchmark from Figure 1. Size shift uses four graph-classification datasets (D&D, NCI1, NCI109, PROTEINS) with GCN, GIN, and PNA backbones. Concept and covariate shift use the GOOD benchmark suite (GOODCMNIST-color, GOODMotif-basis, GOODMotif-size, GOODSST2-length). The baselines are Vanilla, Deep Ensembles (DEns), temperature scaling (Temp), and Monte Carlo Dropout (MCD). Start with size shift.

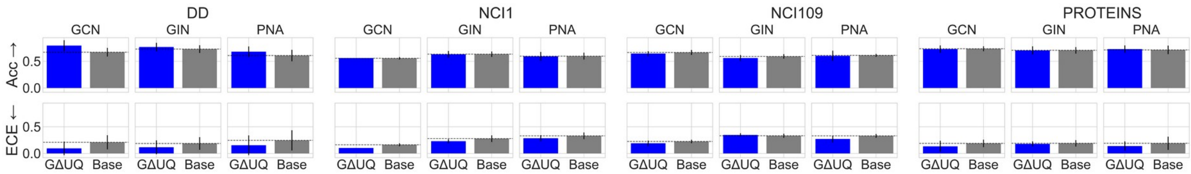

Figure 4. Predictive uncertainty under size distribution shift. Each pair compares G-ΔUQ (blue) against a no-anchoring baseline (Base, gray), with accuracy on top (higher is better) and ECE on the bottom (lower is better). Across D&D, NCI1, NCI109, and PROTEINS and across the GCN, GIN, and PNA backbones, stochastic centering almost always lowers calibration error while maintaining or improving accuracy.

For concept and covariate shift, the authors line G-ΔUQ up directly against the other single-model UQ methods.

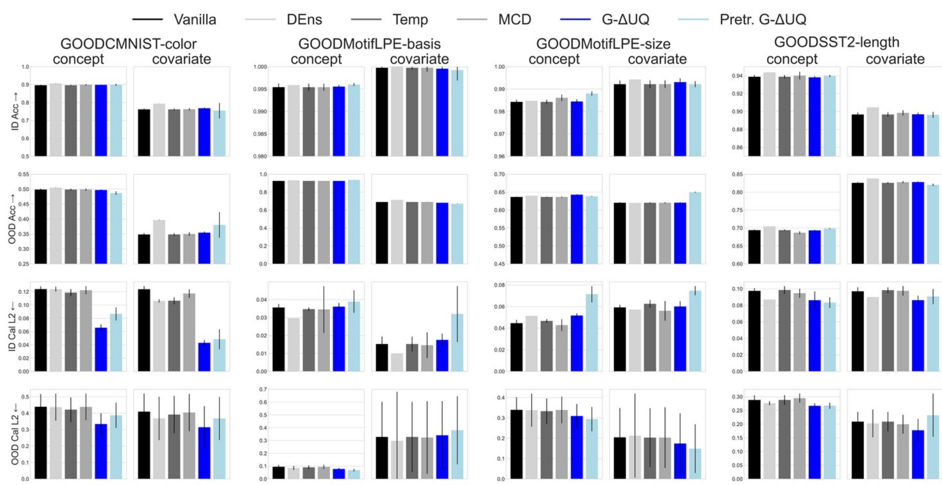

Figure 5. Predictive uncertainty under concept and covariate shift. Spanning the four GOOD datasets, each split into concept and covariate shift, the four rows are in-distribution accuracy (ID Acc), out-of-distribution accuracy (OOD Acc), in-distribution calibration (ID Cal L2), and out-of-distribution calibration (OOD Cal L2). Stochastic anchoring keeps ID and OOD accuracy competitive while improving calibration, and the advantage is clearest against the other single-model UQ methods.

One line summarizes all three figures: whether the shift comes from structure, size, or concept and covariate, G-ΔUQ delivers more trustworthy confidence than the other single-model methods, with accuracy essentially untouched.

What better confidence actually buys you

Calibration is only a means. The real payoff is whether those calibrated confidence estimates, when fed into downstream safety tasks, produce better results. The authors examine two: generalization-gap prediction and out-of-distribution detection.

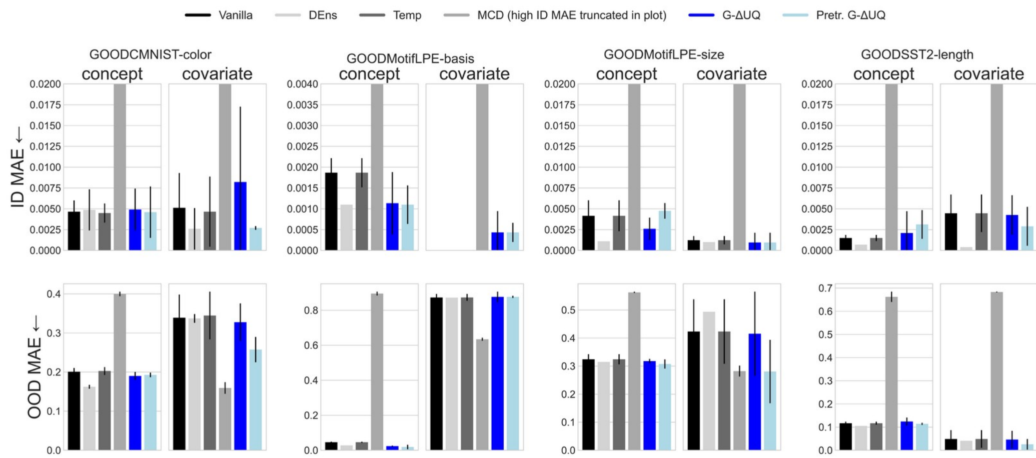

Figure 6. Generalization-gap prediction. Confidence scores from each method are used to predict the generalization error, reported as mean absolute error, MAE (lower is better), with in-distribution on top and out-of-distribution on the bottom. No single method dominates, but stochastic anchoring is highly competitive and yields among the lowest MAE of the single-model UQ estimators; the pretrained G-ΔUQ variant is especially effective, even outperforming the end-to-end version.

Generalization-gap prediction, estimating how much a model will degrade without peeking at test labels, is a task that has barely been explored for graphs. That the pretrained variant beats the end-to-end one here is a practical signal: you do not have to retrain from scratch to get good uncertainty.

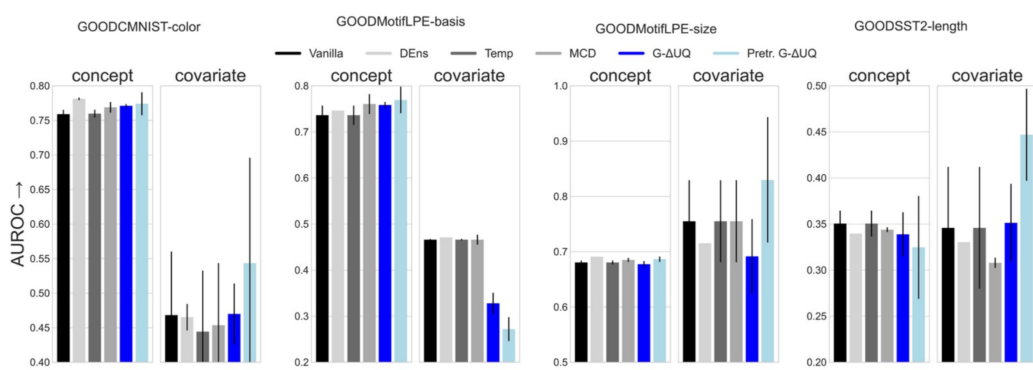

Figure 7. OOD detection. AUROC for detecting out-of-distribution samples (higher is better). Under concept shift, the G-ΔUQ variants are competitive with all baselines, including Deep Ensembles. Under covariate shift, with the sole exception of GOODMotif-basis, pretrained G-ΔUQ delivers significant gains over every baseline, including end-to-end G-ΔUQ.

Taken together, the two downstream tasks reveal a recurring pattern: under covariate shift, the hardest case, pretrained G-ΔUQ is frequently the strongest of the lot. The cheapest way to use the method, freezing the backbone and training only the head, is often also the best.

A key ablation: where to anchor

If anchoring can go early, middle, or late, does the choice matter? The authors ablate it on D&D, the dataset with the most severe size shift. Anchoring after the READOUT (the last layer) makes both accuracy and calibration improve markedly as the network deepens, while earlier-layer anchoring converges less cleanly. That yields a practical recommendation: when a model is expected to face size shift, anchor at the last layer. A neural tangent kernel (NTK) analysis further confirms that applying constant shifts at intermediate layers genuinely changes the function the GNN represents, rather than acting as a trivial reparameterization that cancels out.

Takeaway and boundaries

The core message is plain: do not expect a larger, more expressive graph neural network to hand you trustworthy confidence: under distribution shift, it will not. The move that works is to carry the principle of stochastic anchoring over to graphs. G-ΔUQ is a flexible, scalable, single-model framework that produces reliable, well-calibrated uncertainty across structural, size, concept, and covariate shifts, and across the three safety tasks of calibration, generalization-gap prediction, and OOD detection.

And its pretrained variant makes all of that nearly free to bolt onto a model you have already trained. The honest boundary: these conclusions rest on the paper's specific datasets, backbones, and shift setups, and the best anchoring layer depends on the task (last-layer is recommended for size shift). Carrying the method to a new setting still deserves the same kind of validation the authors ran here.